Week 8: Advanced TypeScript Concepts

FH University of Applied Sciences
TECHNIKUM WIEN

Week 8: Advanced TypeScript Concepts

Topics for This Week

1. Advanced TypeScript Features
2. Modular Web Application Design
3. Building a Modular Chat Application
4. Extending the Application in Future Weeks

1. Advanced TypeScript Features

Type Guards

- Ensure runtime type checks with custom logic.
- Useful when working with `unknown` or `any` types.

```
function isString(value: unknown): value is string {
    return typeof value === "string";
}

const input: unknown = "Hello";
if (isString(input)) {
    console.log(input.toUpperCase()); // Safe to call string methods
}
```

Mapped Types

- Dynamically create new types based on existing ones.

```
type ReadOnly<T> = {
    readonly [K in keyof T]: T[K];
};

interface User {
    id: number;
    name: string;
}

const user: ReadOnly<User> = { id: 1, name: "Alice" };
// user.id = 2; // Error: Cannot assign to 'id' because it is a read-only property
```

Utility Types

- Built-in TypeScript helpers.

Partial

Make all properties optional.

```
type PartialUser = Partial<User>;
const partialUser: PartialUser = { name: "Alice" };
```

Record

Define a key-value structure.

```
type Scoreboard = Record<string, number>;
const scores: Scoreboard = { Alice: 10, Bob: 15 };
```

Decorators (Experimental)

- Add metadata or behavior to classes and methods.

```
function Log(target: any, key: string) {
    console.log(`${key} was accessed.`);
}

class Example {
    @Log
    someMethod() {}
}
```

2. Modular Web Application Design

Principles of Modular Design

1. **Separation of Concerns**:
   - Each module has a specific responsibility.
2. **Reusability**:
   - Modules can be reused across applications.
3. **Extensibility**:
   - Easy to add new features without modifying existing code.

Chat Application: Core Modules

1. **UserManager**:
   - Handles user registration, login, and management.
2. **MessageManager**:
   - Sends, receives, and stores chat messages.
3. **ChatUI**:
   - Renders the user interface for chat interactions.

3. Building a Modular Chat Application

Step 1: Define Interfaces

```
interface ChatMessage {
    sender: string;
    content: string;
    timestamp: Date;
}

interface User {
    id: number;
    name: string;
}
```

Step 2: Implement UserManager

```
export class UserManager {
    private users: User[] = [];

    registerUser(name: string): User {
        const user = { id: this.users.length + 1, name };
        this.users.push(user);
        return user;
    }

    getUserById(id: number): User | undefined {
        return this.users.find(user => user.id === id);
    }
}
```

Step 3: Implement MessageManager

```
import axios from 'axios';

export class MessageManager {
    private messages: ChatMessage[] = [];

    async fetchMessages(): Promise<ChatMessage[]> {
        const response = await axios.get<ChatMessage[]>('https://mockapi.example.com/messages');
        this.messages = response.data;
        return this.messages;
    }

    addMessage(message: ChatMessage): void {
        this.messages.push(message);
    }
}
```

Step 4: Implement ChatUI

```
import { MessageManager } from './MessageManager';

export class ChatUI {
    constructor(private messageManager: MessageManager) {}

    async render(container: HTMLElement): Promise<void> {
        const messages = await this.messageManager.fetchMessages();
        container.innerHTML = messages.map(msg => `
            <div class="message">
                <strong>${msg.sender}:</strong> ${msg.content} <em>${msg.timestamp}</em>
            </div>
        `).join('');
    }
}
```

4. Extending the Application

Future Enhancements

1. **Real-Time Updates**:
   - Use WebSockets for real-time messaging.
2. **Authentication**:
   - Add user login/logout functionality.
3. **Group Chats**:
   - Allow multiple users to join a chat room.
4. **Persistent Data**:
   - Save messages and user data to a database.

Weekly Exercise

Build a Modular Chat Application

1. Implement the core modules:
    - `UserManager`
    - `MessageManager`
    - `ChatUI`
2. Use mock data to simulate messages.
3. Fetch messages from a mock REST API.

Summary and Q&A

Key Takeaways

- Advanced TypeScript features like Type Guards, Utility Types, and Decorators.
- Principles of modular web application design.
- Practical application through building a chat program.

Questions?